

Understanding Keys and Cardinalities: The Foundation of Data Modeling

Introduction

Imagine you're storing customer transaction data. You have a single massive table with every transaction ever recorded, and in that table, you've stored the customer's name, city, phone number, date of birth—everything—alongside each transaction.

Now imagine that customer "Joan Roche" made 5 purchases. Her name, address, and birthday appear 5 times in your table. One day, Joan moves to a new city. Do you update her address in 1 place? 5 places? All 10 places across your dataset?

What if you miss one? Now you have inconsistent data—Joan lives in two cities at once, depending on which row you look at.

This is the problem that **keys and cardinalities** solve. They're the rules that let you break one messy table into organized, connected tables. And they're essential to becoming a data professional.

Why This Matters

Before you normalize data in Power Query, you need to understand **how** tables relate to each other. Before you can understand relationships, you need to understand keys. Before you can use keys correctly, you need to know which ones to use and why.

Think of keys and cardinalities as the language of databases. If you can't speak the language, you'll build models that work—but barely. You'll update data in the wrong places, create relationships that break, and spend hours debugging logic that should have taken 10 minutes.

By the end of this note, you'll understand exactly how to structure your data so it's clean, consistent, and performant.

Part 1: Keys—The Foundation

A **key** is a column (or group of columns) that identifies rows in a table. But not all keys are the same. Let's break them down.

Primary Key: The ID Card

What it is: A column that uniquely identifies each row in a table. No two rows can have the same value, and it can never be empty (null).

Why it matters: The primary key is how you refer to a specific row. It's the row's ID card. Without it, how do you know you're talking about the right row?

In your data:

Look at your transactions table. Each transaction has a `TransactionID`: 3660001, 3660011, 3660012, etc. These are unique. No `TransactionID` appears twice. This is your primary key.

```
TransactionID = 3660001 refers to exactly one transaction:  
Order 366000, Line 1, Customer 265598, Store 10,  
Product 1304, Quantity 1, Date 1/1/2016
```

Similarly, in your Customers dimension, `CustomerID` is the primary key. Each customer has one ID: 265598, 1269051, 266019, etc.

The rule: Every dimension table has one. Every fact table has one. Non-negotiable.

Foreign Key: The Link

What it is: A column in one table that references the primary key of another table. It's how tables "talk" to each other.

Why it matters: Foreign keys create relationships. They prevent you from inserting garbage data (like a `CustomerID` that doesn't exist). They maintain **referential integrity**—a fancy term for "your data makes sense."

In your data:

In your Transactions table, you have:

- `CustomerID = 266019` → This points to the Customers table where `CustomerID = 266019`
- `StoreID = 10` → This points to the Stores table where `StoreID = 10`
- `ProductID = 1304` → This points to the Products table where `ProductID = 1304`

Each of these is a **foreign key**. It links the transaction to a customer, a store, and a product.

The benefit: Instead of storing Joan Roche's entire profile (name, address, phone, birthday) in every row where she appears, you store it once in the Customers table. In Transactions, you just store her ID: 266019. Much cleaner.

Surrogate Key: The Secret ID

What it is: An artificial column, usually an auto-incrementing integer, used as the primary key. It has no business meaning—it's just a number the system generates.

Why it matters: Performance. Speed. Stability.

Integers are faster to store and join on than long text strings. If your business key changes (which it will), your surrogate key doesn't. Your relationships stay solid.

Example:

Instead of using `CustomerID` as your primary key in the Customers table, you could create a new column:

```
CustomerSurrogateKey = 1, 2, 3, 4, 5... ← Primary Key
CustomerID = 265598, 1269051, 266019... ← Alternate Key (still unique, still useful)
CustomerName = Tyler Vaught, Frank Upchurch, Joan Roche...
```

In Transactions, you'd reference `CustomerSurrogateKey` instead of `CustomerID`:

```
Transactions:
TransactionID | CustomerSurrogateKey | StoreKey | ProductKey | ...
3660001      | 1                    | 5        | 100        | ...
3660011      | 2                    | 1        | 50         | ...
3660012      | 2                    | 1        | 120        | ...
```

When to use: Always. Surrogate keys are standard practice in professional data modeling.

Alternate Key: The Backup ID

What it is: A column that *could* be the primary key but isn't. It's still unique, still valuable, but you chose something else as PK.

Why it matters: You keep the original business ID for reference, auditing, and queries. But you use a surrogate key for relationships.

In your data:

If you create a surrogate `CustomerSurrogateKey`, then `CustomerID` becomes an **alternate key**. It's still unique per customer, still identifies them, but it's no longer the "official" ID that Transactions references.

The benefit: You can change your ID scheme without breaking all your relationships. If the business decides to use email instead of CustomerID, you update the alternate key. Your foreign keys (based on surrogate) stay intact.

Candidate Key: The Shortlist

What it is: Any column (or combination of columns) that *could* work as a primary key. Multiple candidates can exist; you choose one.

Why it matters: It forces you to think about what uniquely identifies a row. It helps you design better.

In your data:

In Customers, which could be the primary key?

- `CustomerID` (unique, never null) ✓
- `CustomerName` (not unique—multiple people have the same name) ✗
- `CustomerEmail` (if you had it, unique) ✓
- `CustomerPhone + CustomerName` (combination is unique) ✓

All three of the ✓ examples are **candidate keys**. You evaluate them and choose one. The others become alternate keys.

Composite Key: The Power Couple

What it is: A primary key made of 2 or more columns. No single column is unique; the *combination* is.

Why it matters: It captures the business structure. Some entities are naturally multi-part.

In your data:

Look at your transactions. You have `OrderNumber` and `LineItem`:

```
OrderNumber = 366000, LineItem = 1  ← Transaction 1
OrderNumber = 366001, LineItem = 1  ← Transaction 2 (same order, different line)
OrderNumber = 366001, LineItem = 2  ← Transaction 3 (same order, different line)
OrderNumber = 366002, LineItem = 1  ← Transaction 4
```

Neither `OrderNumber` nor `LineItem` alone is unique. But together, they form a unique pair. You could use `OrderNumber + LineItem` as your composite primary key instead of `TransactionID`.

When to use: For detail/line-item tables where multiple rows belong to one header.

Part 2: Cardinality—The Relationship Rules

Cardinality describes how many rows in one table relate to how many rows in another. It's the "math" of relationships.

There are three types. Master them.

One-to-One (1:1)

The rule: One row in Table A relates to exactly one row in Table B, and vice versa.

Picture it:

Table A

Person
Alice
Bob
Carol

Table B

Passport
Passport #12345
Passport #67890
Passport #11111

1:1→

One person has one passport. One passport belongs to one person.

In your data:

A store has one set of store details. Store 10 (Canada, Nunavut, 1210 sqm, opened 4/4/2015) relates to exactly one record. A 1:1 relationship.

When you see it: Less common than 1:N. Usually means you could merge the tables. In production, you'd only keep 1:1 if:

- Table B has many nulls (not all stores have a warehouse size, for example)
- Table B changes frequently (don't want to update the main table)
- You're planning to add more columns to one side later

One-to-Many (1:N)

The rule: One row in Table A relates to many rows in Table B, but each row in Table B relates to only one row in Table A.

Picture it:

Table A

Customer
Alice
Bob
Carol

Table B

Transactions
Txn 1 (Alice)
Txn 2 (Alice)
Txn 3 (Bob)
Txn 4 (Carol)
Txn 5 (Carol)
Txn 6 (Carol)

1:N→

→

One customer can have many transactions.
But each transaction belongs to exactly one customer.

In your data:

Customer 266019 (Joan Roche) appears in 3 transactions (lines 4, 5, 6 of your table). Customer 1269051 (Frank Upchurch) appears in 2 transactions (lines 2, 3). One customer, many transactions. Classic 1:N.

Similarly:

- Store 10 relates to many transactions (any transaction that happened at Store 10)
- Product 1048 relates to many transactions (every time someone bought that product)

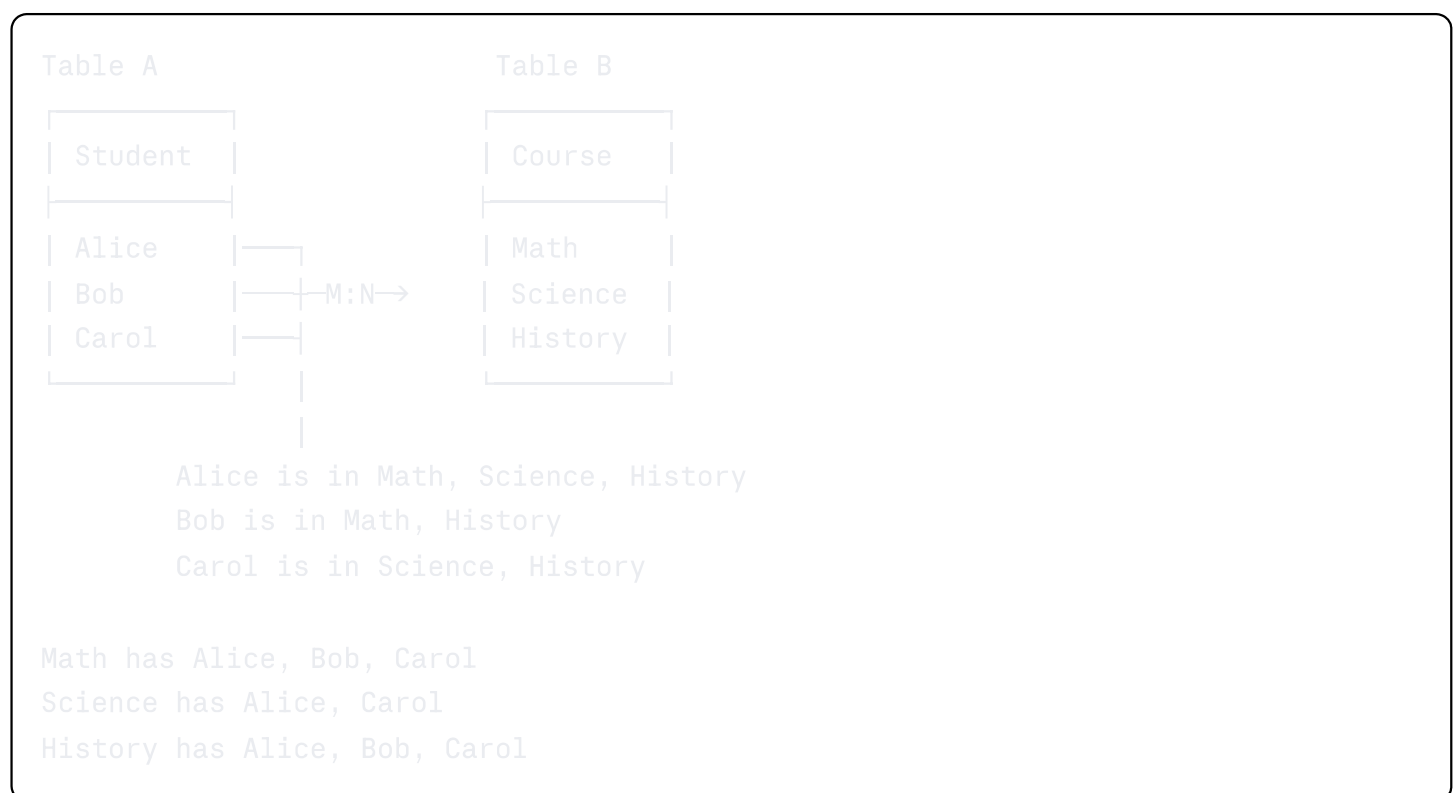
When you see it: Constantly. This is the bread and butter of data modeling. Any time you have a parent-child relationship, it's 1:N. Orders have line items. Customers have transactions. Stores have sales. All 1:N.

Why it matters: It lets you store data once and reference it many times. Joan's profile (name, address, birthday) is stored once. Her ID is referenced in 3 transactions. No duplication.

Many-to-Many (M:N)

The rule: Many rows in Table A relate to many rows in Table B.

Picture it:



In your data:

Here's where it gets interesting. You have Products and Customers.

- Customer 266019 buys Products 1304, 1048, 2007 (3 different products)
- Customer 1269051 buys Products 1048, 2007 (2 different products)

- Product 1048 is bought by Customers 1269051, 266019, and others

This is M:N. Many customers buy many products.

The problem with M:N:

You can't store M:N directly in two tables without creating a mess. If you tried:

✗ WRONG: Storing M:N directly

Customers Table:

CustomerID	ProductID
266019	1304, 1048, 2007
1269051	1048, 2007

← This is messy. Can't query it properly.

Or:

✗ WRONG: Duplicating customers for each product

CustomerID	ProductID
266019	1304
266019	1048
266019	2007
1269051	1048
1269051	2007

↑ Not actually wrong, but this IS your Transactions table!

The solution: Bridge Table (Junction Table)

You introduce a third table in the middle to break M:N into two 1:N relationships:

Customers (1) —1:N→ Transactions —N:1→ Products

One customer has many transactions.
Each transaction belongs to one customer.
Each transaction involves one product.
But one product can appear in many transactions.

Result: M:N relationship is represented cleanly!

In your data:

Your Transactions table is the bridge table. It connects Customers and Products through individual purchase events. Brilliant design.

When you see it: Student ↔ Courses, Products ↔ Suppliers, Employees ↔ Projects. Any "both can be many" scenario needs a bridge.

Part 3: Putting It Together—Your Data Model

Let's apply everything you just learned.

The Problem: Your Current Table

You have one massive denormalized table with 30 columns. Let's count the duplication:

```
Customer 266019 (Joan Roche) appears 3 times.  
Her entire profile (name, gender, city, state, zip, country, continent, DOB)  
is repeated 3 times.  
  
Product 1048 appears 2 times.  
Its entire profile (name, brand, color, cost, price, category, subcategory)  
is repeated 2 times.  
  
Store 10 appears 1 time.  
Store 0 (Online) appears 7+ times.  
Everything is repeated.
```

Cost of duplication:

- Storage space wasted (x3, x2, x7+)
- Update nightmare (change one fact, need to update multiple rows or data gets inconsistent)
- Insert problem (can't add a store without a transaction)
- Delete problem (delete a transaction, lose customer info? Or keep orphaned rows?)

The Solution: Star Schema

Break it into 6 tables:

FACT TABLE (the event/transaction):

```
Transactions  
├ TransactionID (PK)  
├ OrderNumber + LineItem (Composite Key, Alternate PK)  
├ OrderDate  
├ DeliveryDate  
├ Quantity  
├ CustomerID (FK) → Customers  
├ StoreID (FK) → Stores  
└ ProductID (FK) → Products
```

DIMENSION TABLES (the "what/who/where"):

Customers

- └ CustomerID (PK / Surrogate)
- └ CustomerGender
- └ CustomerName
- └ CustomerCity
- └ CustomerStateCode
- └ CustomerState
- └ CustomerZip
- └ CustomerCountry
- └ CustomerContinent
- └ CustomerDOB

Stores

- └ StoreID (PK / Surrogate)
- └ StoreCountry
- └ StoreState
- └ StoreSqMeters
- └ StoreOpenDate

Products

- └ ProductID (PK / Surrogate)
- └ ProductName
- └ ProductBrand
- └ ProductColor
- └ ProductCost
- └ ProductPrice
- └ ProductSubcategoryID (FK) → ProductSubcategories
- └ ProductCategoryID (FK) → ProductCategories

ProductSubcategories

- └ ProductSubcategoryID (PK)
- └ ProductSubcategoryName
- └ ProductCategoryID (FK) → ProductCategories

ProductCategories

- └ ProductCategoryID (PK)
- └ ProductCategoryName

The Relationships (Cardinalities)

Customers (1) ←N┐
└ Transactions (N:M resolved)

Products (1) ←N┐
└ (one transaction, one customer, one product)

Stores (1) ←N┐

ProductCategories (1) ←N- ProductSubcategories (1) ←N- Products (1) ←N- Transactions

Reading the diagram:

- 1 Customer has many Transactions (1:N)
- 1 Store has many Transactions (1:N)
- 1 Product has many Transactions (1:N)
- M Customers relate to M Products through Transactions (M:N resolved)
- 1 Category has many Subcategories (1:N)
- 1 Subcategory has many Products (1:N)

The Benefits

Problem	Before (Denormalized)	After (Normalized)
Storage	Customer repeated 3x	Customer stored once
Updates	Update Joan's address in 3 places or it breaks	Update once, everywhere
Inserts	Can't add a product without a transaction	Add product anytime
Deletes	Delete transaction = lose customer? Or orphaned rows?	Delete freely, no issues
Speed	One huge table, slow	Smaller tables, fast joins
Clarity	Mixed facts & dimensions, confusing	Clean separation, obvious

Part 4: Building Your Model in Power Query

Once you understand keys and cardinalities, normalizing in Power Query becomes straightforward:

1. **Identify dimensions:** Customers, Stores, Products, ProductSubcategories, ProductCategories
2. **Create fact table:** Transactions (with foreign keys)
3. **Build relationships:** One-to-many from each dimension to fact
4. **Add surrogate keys:** Auto-incrementing IDs for performance
5. **Validate cardinalities:** Make sure each relationship matches the rule

You've already learned the hard part. The Power Query part is just mechanics.

Summary: What You Need to Remember

Keys:

- **Primary Key:** Uniquely identifies each row. Every table has one.
- **Foreign Key:** References another table's primary key. Creates relationships.
- **Surrogate Key:** Artificial integer PK. Use always for performance.
- **Alternate Key:** Could have been PK, but wasn't. Keep for reference.
- **Composite Key:** Multiple columns together form the PK. Use for hierarchies.

Cardinalities:

- **1:1:** One row relates to one row. Rare.
- **1:N:** One row relates to many. Most common. Use always.
- **M:N:** Many rows relate to many. Resolve with a bridge table.

In your model:

- Customers, Stores, Products = Dimensions (1:N to Transactions)
- Transactions = Fact (the event)
- Categories, Subcategories = Hierarchies (1:N chains)
- All relationships: 1:N (no M:N, bridge resolves it)

Why this matters:

- Eliminates duplication
- Prevents inconsistency
- Enables fast queries
- Makes updates easy
- Scales to billions of rows
- Becomes second nature with practice

Next Steps

1. **Draw the ERD** in draw.io using what you've learned
2. **Label each key:** PK, FK, Surrogate, Alternate
3. **Mark cardinalities:** 1:1, 1:N, M:N
4. **Build in Power Query:** Create dimensions, build fact, add relationships

5. **Load in Power BI:** Create measures, analyze, present

You're ready. Let's go.